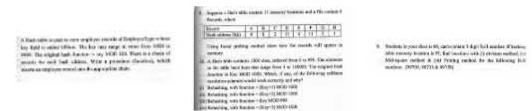


```
# insert_employee_record(emp2, my_hash_table)
# insert_employee_record(emp3, my_hash_table)
# insert_employee_record(emp4, my_hash_table)

# To verify (conceptual):
# print(my_hash_table[34]) # Should contain emp1
# print(my_hash_table[78]) # Should contain emp2
# print(my_hash_table[0]) # Should contain emp3 and emp4 in a list/chain
```

You stopped this response



solve the hashing question (only solve by technique, no need for code)



Okay, I will solve the hashing questions using the specified techniques, without providing code. Let's address each question one by one.

Question 8: Hashing Methods

Given:

- Number of students = 60
- Each student has a 5-digit Roll number.
- Hashing table memory location (table size) = 97
- Roll numbers to find locations for: 99705, 99735, 99759

(i) Division Method

The division method calculates the hash address using the formula: $h(key) = key \pmod{table_size}$

- For Roll number 99705:
 $h(99705) = 99705 \pmod{97} = 16$
- For Roll number 99735:
 $h(99735) = 99735 \pmod{97} = 19$
- For Roll number 99759:
 $h(99759) = 99759 \pmod{97} = 43$

(ii) Mid-square Method

The mid-square method involves squaring the key and then extracting a fixed number of middle digits. The number of digits extracted depends on the table size (97 requires 2 digits, as $97 < 100$).

- For Roll number 99705:
 $99705^2 = 9941088025$
 Taking the middle two digits (e.g., from the central portion of the squared value), let's consider '08'.
 $h(99705) = 08 \pmod{97} = 8$
- For Roll number 99735:
 $99735^2 = 9947072225$
 Taking the middle two digits, let's consider '72'.
 $h(99735) = 72 \pmod{97} = 72$
- For Roll number 99759:
 $99759^2 = 9951850881$
 Taking the middle two digits, let's consider '85'.
 $h(99759) = 85 \pmod{97} = 85$

(iii) Folding Method

The folding method involves dividing the key into parts, adding these parts, and then applying a modulo operation by the table size.

- For Roll number 99705:
 Divide into parts: 99, 70, 5
 $\text{Sum} = 99 + 70 + 5 = 174$
 $h(99705) = 174 \pmod{97} = 77$
- For Roll number 99735:
 Divide into parts: 99, 73, 5
 $\text{Sum} = 99 + 73 + 5 = 177$
 $h(99735) = 177 \pmod{97} = 80$
- For Roll number 99759:
 Divide into parts: 99, 75, 9
 $\text{Sum} = 99 + 75 + 9 = 183$
 $h(99759) = 183 \pmod{97} = 86$

Question 9: Linear Probing Method

Given:

- Hash table with 11 memory locations (indexed 0 to 10).
- 8 Records with their initial Hash addresses $H(k)$:

Record	A	B	C	D	E	F	G	H
Initial Hash Address	4	8	2	11	4	11	5	1

We use linear probing to resolve collisions, meaning we check $(H(k) + i) \pmod{11}$ for $i = 0, 1, 2, \dots$. Note that $11 \pmod{11} = 0$.

Let's represent the hash table as an array of 11 slots, initially empty:

[_, _, _, _, _, _, _, _, _, _, _] (Indices 0-10)

1. **Record A, $H(A) = 4$:**

Place A at index 4.

[_, _, _, _, A, _, _, _, _, _, _]

2. **Record B, $H(B) = 8$:**

Place B at index 8.

[_, _, _, _, A, _, _, B, _, _]

3. **Record C, $H(C) = 2$:**

Place C at index 2.

[_, _, C, _, A, _, _, B, _, _]

4. **Record D, $H(D) = 11 \Rightarrow 0$:**

Place D at index 0.

[D, _, C, _, A, _, _, B, _, _]

5. **Record E, $H(E) = 4$:**

Index 4 is occupied by A.

Next available slot: $(4 + 1) \pmod{11} = 5$. Index 5 is empty.

Place E at index 5.

[D, _, C, _, A, E, _, B, _, _]

6. **Record F, $H(F) = 11 \Rightarrow 0$:**

Index 0 is occupied by D.

Next available slot: $(0 + 1) \pmod{11} = 1$. Index 1 is empty.

Place F at index 1.

[D, F, C, _, A, E, _, B, _, _]

7. **Record G, $H(G) = 5$:**

Index 5 is occupied by E.

Next available slot: $(5 + 1) \pmod{11} = 6$. Index 6 is empty.

Place G at index 6.

[D, F, C, _, A, E, G, B, _, _]

8. **Record H, $H(H) = 1$:**

Index 1 is occupied by F.

Next available slot: $(1 + 1) \pmod{11} = 2$. Index 2 is occupied by C.

Next available slot: $(2 + 1) \pmod{11} = 3$. Index 3 is empty.

Place H at index 3.

[D, F, C, H, A, E, G, B, _, _]

Final appearance of records in memory:

Index	0	1	2	3	4	5	6	7	8	9	10	
:----	:-	:-	:-	:-	:-	:-	:-	:-	:-	:-	:-	
Record	D	F	C	H	A	E	G		B			

Question 10: Collision Resolution Schemes

Given:

- Hash table contains 1000 slots, indexed from 0 to 999.
- Keys range from 1 to 100000.
- Original hash function: $H(\text{key}) = \text{Key} \pmod{1000}$

Original hash function: $H(key) = key \pmod{1000}$

For a linear probing-like rehashing function to work correctly (i.e., to guarantee visiting all slots in the table in the worst case, if necessary), the step size (the constant added in the probing sequence) must be relatively prime to the table size. Here, table size = 1000.

(i) **Rehashing, with function** $= (Key + 1) \pmod{1000}$

* Step size = 1.

* $gcd(1, 1000) = 1$.

* **Works correctly.** Because the step size is relatively prime to the table size, this function can visit every slot in the table.

(ii) **Rehashing, with function** $= (Key + 2) \pmod{1000}$

* Step size = 2.

* $gcd(2, 1000) = 2$.

* **Does not work correctly.** Since the greatest common divisor is 2 (not 1), this function will only visit a subset of the slots (specifically, only even-indexed slots if the initial hash is even, or odd-indexed slots if the initial hash is odd). It cannot probe all locations.

(iii) **Rehashing, with function** $= Key \pmod{998}$

* This is not a typical linear probing increment. It appears to be an entirely different hash function, or a modulo operation on the key itself, not an increment for probing. If interpreted as a step size, then the step would be 998, or $998 \pmod{1000} = 998$.

* $gcd(998, 1000) = 2$.

* **Does not work correctly.** Regardless of the exact interpretation, if 998 is the step, it's not relatively prime to 1000. It would fail to visit all slots.

(iv) **Rehashing, with function** $= (Key + 3) \pmod{1000}$

* Step size = 3.

* $gcd(3, 1000) = 1$.

* **Works correctly.** Because the step size is relatively prime to the table size, this function can visit every slot in the table.

Question (No number, related to employee records)

Given:

- Hash table to store employee records of `EmployeeType`.
- Key field is `IdNum`, ranging from 1000 to 9999.
- Original hash function: $H(key) = key \pmod{100}$.
- There is a chain of records for each hash address (implying separate chaining).

Procedure for inserting an employee record into the appropriate chain (conceptual steps):

1. **Extract Key:** Get the `IdNum` from the `EmployeeType` record that needs to be inserted. Let's call this `key`.
2. **Calculate Hash Address:** Apply the hash function to the `key` to find the initial hash address (index) in the hash table. `hash_address = key % 100`
3. **Navigate to Chain:** Go to the slot (index) in the hash table corresponding to `hash_address`.
4. **Insert into Chain:** Add the `EmployeeType` record to the linked list (or other chain data

structure) present at that `hash_address`. In separate chaining, new elements are typically added to the end of the chain, but can also be added to the beginning for simplicity. There's no need to check for existing duplicates in this basic insertion procedure unless specified.